

รายงานการจัดทำ PREEMPT_RT Kernel (Real-Time) สำหรับ Raspberry Pi 4

เวอร์ชัน Kernel ต้นทาง: Linux raspberrypi 6.18.34+rpt-rpi-v8

วัตถุประสงค์: เพื่อปรับแต่งระบบปฏิบัติการให้รองรับการทำงานแบบ Real-Time (Low Latency / Deterministic) ด้วย PREEMPT_RT Patch

ขั้นตอนที่ 1: การเตรียมระบบและติดตั้งเครื่องมือที่จำเป็น (Prerequisites)

ก่อนเริ่มกระบวนการคอมไพล์ จะต้องติดตั้งเครื่องมือและซอฟต์แวร์ที่ใช้ในการตรวจสอบและบิ้ว Kernel เสียก่อน โดยรันคำสั่งต่อไปนี้ผ่าน Command Line:

Bash

```
# อัปเดตรายการแพ็คเกจในระบบให้เป็นเวอร์ชันล่าสุด
```

```
sudo apt update
```

```
# ติดตั้งเครื่องมือสำหรับการคอมไพล์ซอร์สโค้ด
```

```
sudo apt install git bc bison flex libssl-dev make libc6-dev libncurses5-dev libncurses-dev -y
```

ขั้นตอนที่ 2: การดาวน์โหลด Source Code ของ Raspberry Pi Kernel

ทำการดึงซอร์สโค้ดของ Kernel 6.18 จากคลังอย่างเป็นทางการของ Raspberry Pi (GitHub) โดยใช้เทคนิค `--depth=1` เพื่อดาวน์โหลดเฉพาะเวอร์ชันล่าสุด ซึ่งช่วยประหยัดเวลาและพื้นที่จัดเก็บ

Bash

```
# ดาวน์โหลดซอร์สโค้ดเฉพาะสาขา rpi-6.18.y
```

```
git clone --depth=1 -b rpi-6.18.y https://github.com/raspberrypi/linux
```

```
# ย้ายเข้าไปยังไดเรกทอรีของซอร์สโค้ด
```

```
cd linux
```

ขั้นตอนที่ 3: การกำหนดค่าคอนฟิกเริ่มต้น (Configuration)

ทำการตั้งค่าเริ่มต้นให้ตรงกับสถาปัตยกรรมฮาร์ดแวร์ของ Raspberry Pi 4 (64-bit) และเปิดเมนูสำหรับการปรับแต่งเชิงลึก

Bash

```
# เลือกโครงสร้างคอนฟิกพื้นฐานสำหรับบอร์ด Pi 4 (64-bit)
```

```
KERNEL=kernel8
```

```
make bcm2711_defconfig
```

```
# เปิดหน้าต่างกราฟิกเมนู (Ncurses) เพื่อเข้าสู่โหมดปรับแต่งคอร์
```

```
make menuconfig
```

ขั้นตอนที่ 4: การตั้งค่าพารามิเตอร์ผ่าน make menuconfig

เมื่อหน้าต่างเมนูปรากฏขึ้น ให้ดำเนินการปรับแต่งพารามิเตอร์สำคัญ 3 ส่วนเพื่อประสิทธิภาพสูงสุดของ Real-Time Kernel:

1. การเปิดใช้งาน PREEMPT_RT (หัวใจสำคัญ)

เส้นทางเมนู: General setup ---> Preemption Model

ค่าที่ต้องเลือก: เปลี่ยนจากรูปแบบเดิมให้เป็น Fully Preemptible Kernel (Real-Time) (PREEMPT_RT)

เหตุผล: เพื่ออนุญาตให้ Kernel สามารถขัดจังหวะการทำงานของโปรเซสที่มีความสำคัญต่ำกว่าได้ทันทีเมื่อมีงานด่วนเข้ามา

2. การเพิ่มความถี่ของตัวจับเวลาระบบ (Timer Frequency)

เส้นทางเมนู: Kernel Features ---> Timer frequency

ค่าที่ต้องเลือก: เปลี่ยนค่าเป็น 1000 HZ

เหตุผล: เพื่อสั่งให้ระบบทำการตรวจสอบและจัดตารางเวลาทุกๆ 1 มิลลิวินาที

ส่งผลให้แอปพลิเคชันตอบสนองต่อเหตุการณ์ภายนอกได้แม่นยำขึ้น

3. การปิดฟังก์ชันลดความเร็ว CPU (CPU Frequency Scaling)

เส้นทางเมนู: CPU Power Management ---> CPU Frequency scaling

ค่าที่ต้องเลือก: ให้เปลี่ยน Governor หลักให้เป็น Performance (หรือเอาเครื่องหมาย [*] ออกจากระบบสเกลอัตโนมัติ)

เหตุผล: ป้องกันไม่ให้ CPU ลดความเร็วลงในช่วงที่ไม่มีงาน เพราะจังหวะที่ CPU

ต้องเร่งความเร็วกลับขึ้นมาใหม่จะส่งผลให้เกิดความหน่วงสูง (Latency Spike)

หมายเหตุสำหรับ Kernel 6.18 (64-bit): ตัวเลือก Transparent Hugepage Support ภายใต้เมนู Memory Management options จะถูกปิดไว้โดยอัตโนมัติจากโครงสร้างสถาปัตยกรรมของตัว OS อยู่แล้ว จึงไม่จำเป็นต้องเข้าไปแก้ไขในส่วนนี้

เมื่อตั้งค่าเสร็จสิ้น ให้เลื่อนไปที่ปุ่ม < Save > เพื่อบันทึกไฟล์คอนฟิกในชื่อ .config จากนั้นเลือก < Exit > เพื่อกลับสู่หน้าต่าง cmd ปกติ

ขั้นตอนที่ 5: กระบวนการคอมไพล์และการติดตั้ง (Compilation & Installation)

รันคำสั่งเพื่อเริ่มกระบวนการบีวตัว Kernel, โมดูลที่เกี่ยวข้อง และสร้างไฟล์ระบบขึ้นมาใหม่ (ขั้นตอนนี้นับบอร์ด Pi 4 จะใช้เวลาประมาณ 1 ถึง 2 ชั่วโมง แนะนำให้ติดตั้งพัดลมระบายความร้อน)

Bash

```
# สั่งบีว Kernel โดยใช้ฮูมพลัง CPU ทั้ง 4 คอร์ของ Pi 4 พร้อมกัน
```

```
make -j4 Image modules dtbs
```

```
# ติดตั้งโมดูลต่างๆ ลงไปยังระบบปฏิบัติการ
```

```
sudo make modules_install
```

```
# คัดลอก Kernel ตัวใหม่ และอุปกรณ์เสริม (Device Tree) ไปยังพาร์ทิชัน Boot Firmware
```

```
sudo cp arch/arm64/boot/Image /boot/firmware/$KERNEL.img
```

```
sudo cp arch/arm64/boot/dts/broadcom/*.dtb /boot/firmware/
```

```
sudo cp arch/arm64/boot/dts/overlays/*.dtb* /boot/firmware/overlays/
```

```
sudo cp arch/arm64/boot/dts/overlays/README /boot/firmware/overlays/
```

```
# ทำการรีบูตเครื่องเพื่อให้ระบบโหลด Kernel ตัวใหม่ขึ้นมาทำงาน
```

```
sudo reboot
```

ขั้นตอนที่ 6: การทดสอบและการตรวจสอบผลลัพธ์ (Verification)

เมื่อเครื่อง Raspberry Pi 4 ทำการรีบูตกลับมาแล้ว ให้เปิด Command Line อีกครั้งและพิมพ์คำสั่งเพื่อตรวจสอบการทำงาน:

Bash

```
uname -a
```

ตัวอย่างผลลัพธ์ที่สำเร็จ:

Plaintext

```
Linux raspberrypi 6.18.34-rt-rpt-v8 #1 SMP PREEMPT_RT... aarch64 GNU/Linux
```

ขั้นตอนที่ 7: การล็อกคอร์ CPU (CPU Isolation) สำหรับงาน Real-Time

ขั้นตอนหลังจากทำการคอมไพล์และคัดลอกไฟล์ Kernel เสร็จเรียบร้อยแล้ว (ก่อนสั่ง sudo reboot)

ให้ทำการแก้ไขไฟล์คอนฟิกการบูตเพื่อกำหนดสิทธิ์การใช้คอร์ของ CPU

1. เปิดไฟล์คอนฟิกการบูต cmdline.txt ขึ้นมาแก้ไข

Bash

```
sudo nano /boot/firmware/cmdline.txt
```

2. เพิ่มคำสั่งล็อกคอร์ CPU

เมื่อไฟล์เปิดขึ้นมา คุณจะเห็นข้อความยาวๆ อยู่ในบรรทัดเดียวกันทั้งหมด (ห้ามกดขึ้นบรรทัดใหม่เด็ดขาด)

ให้ใช้ปุ่มลูกศรเลื่อนไปที่ท้ายสุดของบรรทัด เคาะสเปซบาร์ (Space) 1 ครั้ง แล้วพิมพ์คำสั่งต่อไปนี้ต่อท้ายลงไป:

Plaintext

```
isolcpus=1,2,3 nohz_full=1,2,3 rcu_nocbs=1,2,3
```

ความหมายของพารามิเตอร์:

isolcpus=1,2,3 : สั่งให้ Linux Kernel แยกคอร์ 1, คอร์ 2 และคอร์ 3 ออกไป

ไม่ให้โปรเซสทั่วไปของระบบเข้ามาใช้งานโดยพลการ

nohz_full=1,2,3 : ลดการรบกวนจากระบบตัวจับเวลา (Timer Interrupt) ในคอร์ที่ถูกแยกออกไป

เพื่อให้การทำงานในคอร์เหล่านั้นมีความนิ่งและแม่นยำสูงสุด

rcu_nocbs=1,2,3 : ย้ายงานเบื้องหลังของ Kernel (Read-Copy Update callbacks) ออกไปจากคอร์เหล่านี้ เพื่อไม่ให้เกิด Latency Spike

เมื่อพิมพ์เสร็จแล้ว ให้กด Ctrl + O แล้วกด Enter เพื่อบันทึก จากนั้นกด Ctrl + X เพื่อออกจากโปรแกรม nano

แล้วจึงทำขั้นตอนถัดไปคือสั่ง : sudo reboot